

1. Import

import pandas as pd

Description: To use Pandas, you first need to import it. pd is the standard convention.

2. Data Creation

Function	Description	Example
pd.DataFrame()	Create a DataFrame from a dictionary/list	df = pd.DataFrame({"Name":["Ali","Hasan"], "Age":[20,22]})
pd.Series()	Create a 1D array	s = pd.Series([1,2,3])
pd.concat()	Concatenate multiple DataFrames/Series	pd.concat([df1, df2])
pd.merge()	SQL-style join	pd.merge(df1, df2, on="id")
pd.get_dummies()	Convert categorical → one-hot encoding	pd.get_dummies(df["Category"])
pd.cut()	Bin continuous data	pd.cut(df["Age"], bins=3)
pd.qcut()	Quantile-based bins	pd.qcut(df["Age"], q=4)

3. Data Reading (Loading)

Function	Description	Example
pd.read_csv()	Create DataFrame from CSV	df = pd.read_csv("file.csv")

Function	Description	Example
pd.read_excel()	Create DataFrame from Excel	df = pd.read_excel("file.xlsx", sheet_name="Sheet1")
pd.read_json()	Create DataFrame from JSON	df = pd.read_json("file.json")
pd.read_html()	Scrape HTML table	dfs = pd.read_html("file.html")
pd.read_sql()	Create DataFrame from SQL query	df = pd.read_sql(query, conn)
pd.read_parquet()	Read Parquet file	df = pd.read_parquet("file.parquet")
pd.read_pickle()	Read Pickle file	df = pd.read_pickle("file.pkl")

4. Data Writing (Saving)

Function	Description	Example
df.to_csv()	Save as CSV	df.to_csv("file.csv", index=False)
df.to_excel()	Save as Excel	df.to_excel("file.xlsx", index=False)
df.to_json()	Save as JSON	df.to_json("file.json")
df.to_sql()	Save to database	df.to_sql("table_name", conn)
df.to_parquet()	Save as Parquet	df.to_parquet("file.parquet")

5. Data Inspection

Function / Attribute	Description	Example
df.head()	Show first 5 rows	df.head()
df.tail()	Show last 5 rows	df.tail()
df.shape	Number of rows and columns	df.shape
df.columns	List column names	df.columns
df.index	Row index	df.index
df.dtypes	Column data types	df.dtypes
df.size	Total elements	df.size
df.values	Data as NumPy array	df.values
df.ndim	Number of dimensions	df.ndim
df.empty	Check if empty	df.empty
df.info()	Data types + null counts	df.info()
df.describe()	Statistical summary	df.describe()

Function / Attribute	Description	Example
<code>df.sample(n=3)</code>	Random rows	<code>df.sample(3)</code>
<code>df.memory_usage()</code>	Memory usage	<code>df.memory_usage()</code>

6. Data Cleaning

Function	Description	Example
<code>df.isnull()</code>	Check for null values	<code>df.isnull().sum()</code>
<code>df.notnull()</code>	Check for non-null values	<code>df.notnull()</code>
<code>df.dropna()</code>	Drop rows with null values	<code>df.dropna()</code>
<code>df.fillna()</code>	Fill null values	<code>df["Age"].fillna(df["Age"].mean())</code>
<code>df.replace()</code>	Replace values	<code>df.replace("old","new")</code>
<code>df.duplicated()</code>	Check duplicates	<code>df.duplicated()</code>
<code>df.drop_duplicates()</code>	Drop duplicates	<code>df.drop_duplicates()</code>
<code>df.astype()</code>	Change data type	<code>df["Age"].astype("int")</code>

7. Selection & Filtering

Function	Description	Example
<code>df.loc[]</code>	Label-based selection	<code>df.loc[2,"Name"]</code>
<code>df.iloc[]</code>	Index-based selection	<code>df.iloc[2,1]</code>
<code>df.at[]</code>	Access single value	<code>df.at[2,"Name"]</code>
<code>df.iat[]</code>	Fast single value access	<code>df.iat[2,1]</code>
<code>df.query()</code>	SQL-style filtering	<code>df.query("Age>20")</code>
<code>df.filter()</code>	Filter columns/rows	<code>df.filter(items=["Name","Age"])</code>

8. Sorting

Function	Description	Example
<code>df.sort_values()</code>	Sort by column	<code>df.sort_values("Age")</code>
<code>df.sort_index()</code>	Sort by index	<code>df.sort_index()</code>
<code>df.nlargest()</code>	Top n values	<code>df.nlargest(3,"Age")</code>

Function	Description	Example
df.nsmallest()	Smallest n values	df.nsmallest(3,"Age")

9. Column Operations

Function	Description	Example
df.insert()	Insert new column	df.insert(2,"NewCol",[1,2])
df.assign()	Create new column	df.assign(NewCol=lambda x:x.Age+1)
df.drop()	Drop column/row	df.drop("NewCol", axis=1)
df.rename()	Rename column	df.rename(columns={"Name":"FullName"})
df.set_index()	Set column as index	df.set_index("Name")
df.reset_index()	Reset index	df.reset_index()

10. GroupBy & Aggregation

Function	Description	Example
df.groupby()	Create groups	df.groupby("Dept")["Salary"].mean()
df.agg()	Aggregate function	df.agg({"Salary":"sum","Age":"mean"})
df.transform()	Group-wise transformation	df.groupby("Dept")["Salary"].transform("mean")
df.apply()	Apply custom function	df.apply(lambda x:x*2)

11. Reshaping

Function	Description	Example
df.pivot()	Reshape table	df.pivot(index="Dept", columns="Year", values="Salary")
df.pivot_table()	Aggregated pivot	df.pivot_table(index="Dept",

Function	Description	Example
		values="Salary", aggfunc="mean")
df.melt()	Wide → Long	df.melt(id_vars=["Dept"], value_vars=["Salary","Bonus"])
df.stack()	Column → Row	df.stack()
df.unstack()	Row → Column	df.unstack()

12. Statistical Functions

Function	Description	Example
df.mean()	Mean	df["Age"].mean()
df.median()	Median	df["Age"].median()
df.mode()	Most common value	df["Age"].mode()
df.std()	Standard deviation	df["Age"].std()
df.var()	Variance	df["Age"].var()
df.min()	Minimum	df["Age"].min()
df.max()	Maximum	df["Age"].max()
df.sum()	Sum	df["Age"].sum()
df.count()	Non-null count	df["Age"].count()
df.corr()	Correlation	df.corr()

13. Series Functions

Function	Description	Example
s.unique()	Unique values	s.unique()
s.nunique()	Count of unique values	s.nunique()
s.value_counts()	Frequency count	s.value_counts()
s.map()	Map function	s.map(lambda x:x*2)
s.apply()	Apply custom function	s.apply(lambda x:x+1)

14. String Operations

Function	Description	Example
<code>.str.lower()</code>	Lowercase	<code>df["Name"].str.lower()</code>
<code>.str.upper()</code>	Uppercase	<code>df["Name"].str.upper()</code>
<code>.str.strip()</code>	Remove extra spaces	<code>df["Name"].str.strip()</code>
<code>.str.contains()</code>	Text search	<code>df["Name"].str.contains("Ali")</code>
<code>.str.replace()</code>	Replace text	<code>df["Name"].str.replace("Ali", "Kazi")</code>
<code>.str.split()</code>	Split string	<code>df["Name"].str.split(" ")</code>

15. DateTime Operations

Function	Description	Example
<code>.dt.year</code>	Year	<code>df["date"].dt.year</code>
<code>.dt.month</code>	Month	<code>df["date"].dt.month</code>
<code>.dt.day</code>	Day	<code>df["date"].dt.day</code>
<code>.dt.weekday</code>	0=Mon → 6=Sun	<code>df["date"].dt.weekday</code>
<code>.dt.day_name()</code>	Day name	<code>df["date"].dt.day_name()</code>

16. Tips & Rules

- Large datasets → Use vectorized operations, avoid loops.
- Missing values → numeric: mean/median, categorical: mode.
- `apply()` vs `transform()` → Use `transform()` for groupwise calculations.
- Parquet → Faster and memory-efficient.
- Always start with:

```
df.info()
df.describe()
df.isnull().sum()
```